

C++ 프로그래밍

다형성 (Polymorphism)

다형성이란?

같은 인터페이스로 서로 다른 타입의 객체를 동일하게 다루는 능력이다.

종류	결정 시점	메커니즘	예
정적 다형성	컴파일 타임	함수 오버로딩, 템플릿	<code>print(int)</code> VS <code>print(double)</code>
동적 다형성	런타임	<code>virtual</code> 함수 + 포인터/참조	<code>Animal*</code> → <code>Dog::speak()</code>

```
// 정적: 컴파일러가 오버로딩 중 어느 버전을 쓸지 결정
void draw(Circle c)    { /* ... */ }
void draw(Rectangle r) { /* ... */ }

// 동적: 런타임에 실제 객체 타입에 따라 함수 결정
void render(Shape* s) {
    s->draw(); // Circle? Rectangle? - 실행 중에 결정
}
```

동적 다형성의 핵심: 기본 클래스 포인터/참조를 통해 파생 클래스 객체를 다루면, 실제로 어떤 함수가 호출되는지는 런타임에 결정된다.

가상 함수 (Virtual Function)

`virtual` 키워드를 붙이면 파생 클래스의 재정의 함수가 런타임에 호출된다.

virtual 없이 — 정적 바인딩

```
class Animal {
public:
    void speak() const { printf("...\n"); }
};
class Dog : public Animal {
public:
    void speak() const { printf("왈왈!\n"); }
};
class Cat : public Animal {
public:
    void speak() const { printf("야옹!\n"); }
};

Animal* p = new Dog();
p->speak();    // ▲ "..." 출력
              // 포인터 타입(Animal) 기준 결정

delete p;
```

virtual 있음 — 동적 바인딩

```
class Animal {
public:
    virtual void speak() const { printf("...\n"); }
    virtual ~Animal() {}
};
class Dog : public Animal {
public:
    void speak() const override { printf("왈왈!\n"); }
};
class Cat : public Animal {
public:
    void speak() const override { printf("야옹!\n"); }
};

Animal* p = new Dog();
p->speak();    // ✅ "왈왈!" 출력
              // 실제 타입(Dog) 기준 결정

delete p;
```

다형성 활용 예시

기본 클래스 포인터/참조를 통해 파생 클래스 객체를 동일한 코드로 처리한다.

다형성을 활용한 클래스 함수

```
#include <stdio>
#include <string>

class Animal {
public:
    std::string name;
    Animal(const std::string& n) : name(n) {}
    virtual void speak() const { printf("%s: ...\n", name.c_str()); }
    virtual ~Animal() {}
};

class Dog : public Animal {
public:
    Dog(const std::string& n) : Animal(n) {}
    void speak() const override { printf("%s: 왈왈!\n", name.c_str()); }
};

class Cat : public Animal {
public:
    Cat(const std::string& n) : Animal(n) {}
    void speak() const override { printf("%s: 야옹!\n", name.c_str()); }
};
```

메인함수에서의 활용

```
int main() {
    Dog dog("멍멍이");
    Cat cat("냥냥이");
    Dog dog2("바둑이");

    Animal* zoo[] = { &dog, &cat, &dog2 };
    for (Animal* a : zoo) a->speak();
    // 출력: 멍멍이: 왈왈!   냥냥이: 야옹!   바둑이: 왈왈!
}
```

순수 가상 함수 & 추상 클래스

= 0 을 붙이면 구현이 없는 순수 가상 함수가 된다. 하나라도 있으면 추상 클래스다.

추상 클래스 (인터페이스 역할)

```
class Shape {
public:
    // 파생 클래스에서 반드시 구현해야 함
    virtual double area() const = 0;
    virtual double perimeter() const = 0;

    // 일반 가상 함수 - 선택적 오버라이딩
    virtual void print() const {
        printf("넓이=%.2f\n", area());
    }

    virtual ~Shape() {}
};

// Shape s;    // ❌ 추상 클래스는 직접 생성 불가
// new Shape; // ❌
```

용도: 공통 인터페이스를 강제하는 기반 타입

구체 클래스 (Concrete Class)

```
class Circle : public Shape {
    double r;
public:
    Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
    double perimeter() const override { return 2 * 3.14159 * r; }
};

class Rect : public Shape {
    double w, h;
public:
    Rect(double w, double h) : w(w), h(h) {}
    double area() const override { return w * h; }
    double perimeter() const override { return 2 * (w + h); }
};

Circle c(5);
Rect r(3, 4);

Shape* shapes[] = { &c, &r };
for (Shape* s : shapes)
    s->print();    // 각 타입의 area() 호출
```

가상 소멸자 (Virtual Destructor)

기본 클래스 포인터로 delete 할 때, 소멸자가 virtual 이 아니면 파생 클래스 소멸자가 호출되지 않는다.

❌ 가상 소멸자 없음 — 메모리 누수

```
class Base {
public:
    ~Base() { printf("Base 소멸\n"); }
};

class Derived : public Base {
    int* data;
public:
    Derived() : data(new int[100]) {}
    ~Derived() {
        delete[] data;          // 실행되지 않음!
        printf("Derived 소멸\n");
    }
};

Base* p = new Derived();
delete p;
// 출력: "Base 소멸" 만 출력
// Derived::~~Derived 미호출 → 메모리 누수!
```

✅ 가상 소멸자 — 올바른 소멸

```
class Base {
public:
    virtual ~Base() { printf("Base 소멸\n"); }
};

class Derived : public Base {
    int* data;
public:
    Derived() : data(new int[100]) {}
    ~Derived() {
        delete[] data;
        printf("Derived 소멸\n");
    }
};

Base* p = new Derived();
delete p;
// 출력: "Derived 소멸"
//       "Base 소멸"
// 파생 → 기본 순서로 올바르게 소멸!
```

규칙: 다형성으로 사용되는 기본 클래스는 반드시 virtual 소멸자를 선언한다.

override & final

C++11 키워드로, 오버라이딩 실수를 **컴파일 타임에** 잡아낸다.

override

```
class Animal {
public:
    virtual void speak() const {}
    virtual void move() const {}
};

class Dog : public Animal {
public:
    // ✅ 서명이 맞는지 컴파일러가 검증
    void speak() const override {}

    // ❌ 컴파일 오류 - Animal::move(int)는 없음
    // void move(int speed) const override {}

    // ⚠️ override 없으면 숨김(hiding) - 경고 없음
    void move(int speed) const {}
};
```

override 가 없으면 오타나 서명 불일치를 잡지 못한다.

final

```
class Shape {
public:
    virtual double area() const = 0;
};

// Circle::area 이후 오버라이딩 금지
class Circle : public Shape {
    double r = 1.0;
public:
    double area() const override final {
        return 3.14159 * r * r;
    }
};

// ❌ Circle::area는 final이므로 오버라이딩 불가
// class Ellipse : public Circle {
//     double area() const override {}
// };

// 클래스 자체 봉인
class ImmutablePoint final {
    double x, y;
};

// class Sub : public ImmutablePoint {}; // ❌
```

요약

개념	키워드 / 문법	핵심
정적 다형성	오버로딩, 템플릿	컴파일 타임 결정, 오버헤드 없음
동적 다형성	<code>virtual</code> + 포인터/참조	런타임 결정, 확장에 열려있음
가상 함수	<code>virtual void f()</code>	파생 클래스 버전이 런타임에 호출됨
vtable	컴파일러 자동 생성	함수 포인터 배열 — 런타임 디스패치
순수 가상 함수	<code>virtual f() = 0</code>	추상 클래스, 인터페이스 강제
가상 소멸자	<code>virtual ~Base()</code>	다형성 클래스에 필수, 누수 방지
override	<code>void f() override</code>	오버라이딩 서명을 컴파일 타임에 검증
final	<code>void f() final</code>	이후 오버라이딩 금지

동적 다형성의 핵심: 기본 클래스 포인터/참조로 파생 클래스를 다루면 `virtual` 함수 호출은 런타임에 실제 객체 타입이 결정한다.